
```

clc;
clear;
% Input Data format: [From To R X]
linedata = [
    1 2 0.07 0.15;
    1 3 0.06 0.10;
    1 4 0.08 0.25;
    2 3 0.04 0.10;
    3 4 0.04 0.20;
];

n = 4;
bus_type = [1 2 3 3]; % 1=Slack, 2=PV, 3=PQ
Pspec = [0; 0.5; -0.8; -0.6]; % Specified P
Qspec = [0; 0; -0.4; -0.3]; % Specified Q

% Calling Ybus Fucntion
Ybus = formYbus(linedata, n);
B = imag(Ybus);

pv_pq_idx = find(bus_type ~= 1); % Buses 2,3,4 (for P)
pq_idx = find(bus_type == 3); % Buses 3,4 (for Q)

% Create B matrices using Matrix Slicing (Efficient)
Bprime = -B(pv_pq_idx, pv_pq_idx);
Bdoubleprime = -B(pq_idx, pq_idx);

V = [1.05; 1.0; 1.0; 1.0];
delta = zeros(n, 1);
tol = 1e-4;
max_iter = 20;

fprintf('Y-bus Matrix:\n'); disp(Ybus);

% --- ITERATION LOOP ---
fprintf('\nStarting Iterations...\n');

for iter = 1:max_iter
    % 1. Vectorized Calculation of P and Q (No nested loops)
    V_complex = V .* exp(1j * delta);
    S_calc = V_complex .* conj(Ybus * V_complex);
    P_calc = real(S_calc);
    Q_calc = imag(S_calc);

    % 2. Mismatches
    dP = Pspec(pv_pq_idx) - P_calc(pv_pq_idx);
    dQ = Qspec(pq_idx) - Q_calc(pq_idx);

    max_err = max([abs(dP); abs(dQ)]);
    fprintf('Iter %d: Max Error = %.6f\n', iter, max_err);

    if max_err < tol

```

```

        fprintf('\nConverged in %d iterations.\n', iter);
        break;
    end

    % 3. Update Angles (P-theta) using Backslash Operator (\)
    % Solves Ax = b efficiently without calculating inverse
    dTh = Bprime \ (dP ./ V(pv_pq_idx));
    delta(pv_pq_idx) = delta(pv_pq_idx) + dTh;

    % 4. Update Voltages (Q-V)
    % Recalculate Q with new angles for better accuracy
    V_complex = V .* exp(1j * delta);
    Q_calc = imag(V_complex .* conj(Ybus * V_complex));
    dQ = Qspec(pq_idx) - Q_calc(pq_idx);

    dV = Bdoubleprime \ (dQ ./ V(pq_idx));
    V(pq_idx) = V(pq_idx) + dV;
end

% --- RESULTS ---
disp(' '); disp('FINAL BUS DATA:');
disp('Bus      V(pu)      Angle(deg)      P(pu)      Q(pu)');
results = [ (1:n)', V, rad2deg(delta), P_calc, Q_calc ];
disp(results);

disp(' '); disp('LINE FLOWS:');
disp('From      To      P_flow      Q_flow      S_mag');

for k = 1:size(linedata, 1)
    fb = linedata(k, 1);
    tb = linedata(k, 2);
    Z = linedata(k, 3) + 1j*linedata(k, 4);

    Vf = V(fb) * exp(1j*delta(fb));
    Vt = V(tb) * exp(1j*delta(tb));

    I_line = (Vf - Vt) / Z;
    S_line = Vf * conj(I_line);

    fprintf('%d      %d      %.4f      %.4f      %.4f\n', ...
        fb, tb, real(S_line), imag(S_line), abs(S_line));
end

Y-bus Matrix:
    8.1276 -16.4558i   -2.5547 + 5.4745i   -4.4118 + 7.3529i   -1.1611 + 3.6284i
   -2.5547 + 5.4745i    6.0030 -14.0951i   -3.4483 + 8.6207i    0.0000 + 0.0000i
   -4.4118 + 7.3529i   -3.4483 + 8.6207i    8.8216 -20.7813i   -0.9615 + 4.8077i
   -1.1611 + 3.6284i    0.0000 + 0.0000i   -0.9615 + 4.8077i    2.1226 - 8.4361i

Starting Iterations...
Iter 1: Max Error = 0.627737
Iter 2: Max Error = 0.209909
Iter 3: Max Error = 0.044141

```

Iter 4: Max Error = 0.008220
Iter 5: Max Error = 0.001534
Iter 6: Max Error = 0.000287
Iter 7: Max Error = 0.000054

Converged in 7 iterations.

FINAL BUS DATA:

Bus	V(pu)	Angle(deg)	P(pu)	Q(pu)
1.0000	1.0500	0	0.9672	1.1138
2.0000	1.0000	1.2178	0.5000	-0.2557
3.0000	0.9736	-1.5814	-0.7999	-0.4000
4.0000	0.9514	-4.2426	-0.6000	-0.3000

LINE FLOWS:

From	To	P_flow	Q_flow	S_mag
1	2	0.0126	0.3457	0.3459
1	3	0.5631	0.4683	0.7324
1	4	0.3915	0.2998	0.4931
2	3	0.5049	0.0737	0.5103
3	4	0.2285	0.0673	0.2382

Published with MATLAB® R2025a